This code illustrates how to create a new Frequent Flyer member ❶, but it lacks precision. You're assuming that the first name comes before the last name, but this isn't demonstrated explicitly. This sort of detail would be useful for a developer using the `FrequentFlyer` class to implement other features, but it would be of little interest to the business. In other words, it's a low-level technical requirement. Let's see how each step of this process works.

DESCRIBE THE EXPECTED BEHAVIOR AS A FAILING TEST

To illustrate and document this technical behavior more clearly, you start by illustrating what the method used in the step definition code (❶ in the previous code snippet) is expected to do, in the form of a unit test:

```
public class WhenRegisteringANewFrequentFlyerMember {

    @Test
    public void should_be_able_to_create_a_new_member() {
        FrequentFlyer member
            = FrequentFlyer.withFrequentFlyerNumber("123456789")
                           .named("Jill", "Smith");

        assertThat(member.getFirstName()).isEqualTo("Jill");
        assertThat(member.getLastName()).isEqualTo("Smith");
        assertThat(member.getFrequentFlyerNumber()).isEqualTo("123456789");#4
    }
}
```

Specify which business requirement you're working on.

❶ Identify which detailed technical requirement or feature you're illustrating.

❷ Create a new Frequent Flyer member.

❸ Verify the expected outcomes.

This method is more than just a unit test; it's a simple and concise executable specification. You've described the requirement ❶, given an example of how you'd like the `FrequentFlyer` API to work ❷, and described the expected outcomes of this operation ❸.

WRITE JUST ENOUGH CODE TO GET THE TEST TO PASS

You can now write some code for the `FrequentFlyer` class, as you see fit, to make this test pass. An important aspect of both TDD and BDD is trying to write the minimum amount of code that will make the test pass. In practice, this may be as little as a single line of code, or it may be a little more for simpler cases or where boilerplate code can be generated by the IDE.

In this case, you could create a simple domain object with a builder class containing the methods required by the API definition:[2]

```
public class FrequentFlyer {

    private String frequentFlyerNumber;
    private String firstName;
    private String lastName;

    protected FrequentFlyer(String frequentFlyerNumber,
                            String firstName,
                            String lastName) {...}
```

Getters omitted for brevity

Private constructor

[2] You can find the tests and application code discussed in this chapter in the sample code for the chapter.